

平成 2 4 年度

筑波大学大学院博士課程
システム情報工学研究科
コンピュータサイエンス専攻

博士前期課程（一般入学試験 8 月期）

試験問題 基礎科目（情報基礎）

[注意事項]

1. 試験開始の合図があるまで、問題の中を見てはいけません。
2. 解答用紙の定められた欄に、研究科、専攻、受験番号を記入すること。
3. この問題は全部で 8 ページ（表紙を除く）です。1 ～ 4 ページは日本語版、5 ～ 8 ページは英語版です。
4. 解答用紙を 2 枚（罫線有り）、下書き用紙（白紙）を 1 枚配布します。
5. 問題は全部で 2 問あります。問題 I の解答を 1 枚目、問題 II の解答を 2 枚目の解答用紙に、必ず分けて記入すること。
6. 解答を記述するにあたって、問題 I の解答、問題 II の解答であることを、必ず明記すること。

English version is shown on page 5 to 8.

平成 2 3 年 8 月 2 5 日

問題 I の解答は 1 枚目の解答用紙に記入すること。問題 I の解答であることを明記すること。

問題 I C 言語で文字型のデータを保持するスタックを実装することを考える。そのスタックには、以下の操作があるとする。

<code>void init_stack(void);</code>	スタックを空にする。
<code>int push(unsigned char data);</code>	スタックに引数で与えられたデータ <code>data</code> を加える。成功した時には 1, スタックがオーバーフローした時には, -1 を返す。
<code>int pop(void);</code>	スタックからデータを取り出し, それを返す。スタックが空の時(アンダーフローした時)には, -1 を返す。
<code>int depth(void);</code>	スタックに含まれているデータの数を返す。

(1) 次のプログラムを実行した時, 画面にどのような出力がなされるかを示せ。

```
int main(void) {
    init_stack();
    push( '(' );
    push( '(' );
    push( '[' );
    push( '{' );
    printf("%c\n", pop() );
    printf("%c\n", pop() );
    printf("%c\n", pop() );
    printf("%c\n", pop() );
    return( 1 );
}
```

(2) 関数 `balance()` は、スタックを使って文字列の中の括弧の対応がとれているかを判定するプログラムである。引数は、括弧 `'('`, `')'`, `'{'`, `'}'`, `'['`, `']'`, およびその他の文字からなる文字列である。データの末尾には、`null` 文字 `'\0'` が置かれている。たとえば、次のような文字列を引数にとる。

```
unsigned char *input0 = "{a{()}}b";
unsigned char *input1 = "{a{()}}b";
unsigned char *input2 = "[]";
unsigned char *input3 = "[]";
```

関数 `balance()` は、文字列の中で括弧の対応がとれていたら 1 を、取れていなければ 0 を返す。たとえば、上の引数が与えられると、関数 `balance()` は、それぞれ 1, 0, 0, 1 を返す。以下の空欄を埋めて、プログラムを完成させなさい。ただし、この関数ではスタックはオーバーフローしないものとする。

```
int balance( unsigned char *p ) {
    unsigned char c;
    init_stack();
    for( ; (c = *p) != '\0'; p++ ) {
        switch( c ) {
            case '(':
            case '[':
            case '{':
                push( c );
                break;
            case ')':
                if( (A) )
                    return( 0 );
                break;
            case '}':
                if( (B) )
                    return( 0 );
                break;
            case ']':
                if( (C) )
                    return( 0 );
                break;
            default:
                break;
        }
    }
    if( depth() (D) )
        return( 1 );
    else
        return( 0 );
}
```

(3) 以下の関数は、上で述べた操作を持つスタックを配列で実装したものである。空欄を埋めてプログラムを完成させなさい。埋めるべきコードがない部分には、「/**/」と答えなさい。なおこのスタックは、配列の添え字が大きい方から小さい方へ向って伸び、また最大で SIZE 個の要素を保持することができるものとする。

```
#define SIZE 100
unsigned char stack[SIZE];
int sp;

void init_stack(void) {
    sp = SIZE;
}

int push(unsigned char data) {
    if( sp  (E)  ) /* stack overflow */
        return( -1 );
     (F)  ;
     (G)  = data;
     (H)  ;
    return( 1 );
}

int pop(void) {
    unsigned char data;
    if( sp  (I)  ) /* stack underflow */
        return( -1 );
     (J)  ;
    data =  (K)  ;
     (L)  ;
    return( data );
}

int depth(void) {
    return(  (M)  );
}
```

(4) 設問 (3) のスタックの実装は、要素数の最大がコンパイル時に固定されているという制限がある。この制限を克服する手法を1つ説明しなさい。なお、具体的な C 言語によるプログラムを示す必要はない。

問題 II の解答は 2 枚目の解答用紙に記入すること。問題 II の解答であることを明記すること。

問題 II 非負整数の集合を $\mathcal{N} = \{0, 1, 2, \dots\}$ とし、再帰関数 $f: \mathcal{N} \rightarrow \mathcal{N}$ を以下で定義する。

$$f(0) = 0$$

$$f(1) = 1$$

$$f(n) = f(n-1) + f(n-2) \quad (n \geq 2 \text{ のとき})$$

このとき、以下の設問 (1)~(3) から 2 問を選択し、その答えを解答用紙に記入しなさい。

- (1) 3 ビットの非負整数 $(x_2x_1x_0)_2$ を入力し、 $f((x_2x_1x_0)_2) \bmod 8 = (y_2y_1y_0)_2$ を出力する論理回路を設計する。但し、 x と y が整数 ($y \neq 0$) のとき、 $x \bmod y$ は x を y で割った時の余りを表す。このとき、以下の設問に答えなさい。

- (a) 出力 y_2, y_1, y_0 の真理値表を作成しなさい。
- (b) y_2 と y_1 の論理式を積和標準形により示しなさい。
- (c) y_0 のカルノー図を作成し、最簡形の論理式を示しなさい。

- (2) 以下の設問に答えなさい。

- (a) 任意の $n \geq 2$ に対して、 $f(n) < f(n+1)$ であることを、数学的帰納法を用いて証明しなさい。
- (b) 関数 f が全射であるならば、それを証明しなさい。そうでなければ、反例を示しなさい。
- (c) 以下で定義される関数 $g: \mathcal{N} \rightarrow \mathcal{N}$ が単射となる $m \in \mathcal{N}$ を 1 つ示しなさい。

$$g(m) = 4$$

$$g(n) = f(n) \quad (n \neq m \text{ のとき})$$

- (3) 以下の設問に答えなさい。

- (a) 以下の論理式が真であるならば証明しなさい。偽であるならば反例を示しなさい。
 - (i) $\forall n \in \mathcal{N} (\forall m \in \mathcal{N} f(n) = f(m) + f(m+1))$
 - (ii) $\forall n \in \mathcal{N} (\exists m \in \mathcal{N} f(3n) = 2m)$
- (b) 論理式 $\forall n \in M (n = f(n))$ が真となる集合 $M \subset \mathcal{N}$ をすべて示しなさい。

Write the answers to Problem I on the first answer sheet, and clearly label it at the top of the page as "Problem I."

Problem I Suppose we implement a stack that holds characters in the C language. This stack has following operations.

<code>void init_stack(void);</code>	Clear the stack.
<code>int push(unsigned char data);</code>	Add an item data given as an argument onto the stack. If successful, this function returns 1. If the stack overflows, this function returns -1.
<code>int pop(void);</code>	Remove an item from the stack and return the item. If the stack underflows, this function returns -1.
<code>int depth(void);</code>	Return the number of items in the stack.

(1) When the following program is executed, what is the output of the program on the computer screen?

```
int main(void) {
    init_stack();
    push( '(' );
    push( '(' );
    push( '[' );
    push( '{' );
    printf("%c\n", pop() );
    printf("%c\n", pop() );
    printf("%c\n", pop() );
    printf("%c\n", pop() );
    return( 1 );
}
```

(2) The function `balance()` is a program that examines whether the pairs and the orders of parentheses, braces, and brackets are correct in a string by using a stack. The argument of this function is a string of characters '(', ')', '{', '}', '[', ']', and other characters. The argument string ends with a null character, '\0'. Examples of input strings follow.

```
unsigned char *input0 = "{a{()}}b";
unsigned char *input1 = "{a{()}}b";
unsigned char *input2 = "[]";
unsigned char *input3 = "[]";
```

The function `balance()` returns 1 if the pairs and the orders of parentheses, braces, and brackets are correct in the string. Otherwise it returns 0. For example, the function `balance()` returns 1, 0, 0, and 1, respectively for the input strings above. Fill in the boxes below to complete the program. You should assume that the stack never overflows in this function.

```
int balance( unsigned char *p ) {
    unsigned char c;
    init_stack();
    for( ; (c = *p) != '\0'; p++ ) {
        switch( c ) {
            case '(':
            case '[':
            case '{':
                push( c );
                break;
            case ')':
                if( (A) )
                    return( 0 );
                break;
            case '}':
                if( (B) )
                    return( 0 );
                break;
            case ']':
                if( (C) )
                    return( 0 );
                break;
            default:
                break;
        }
    }
    if( depth() (D) )
        return( 1 );
    else
        return( 0 );
}
```

(3) The following functions implement a stack with the operations described above by using an array. Fill in the boxes and complete the program. If a box should be empty, write "/* */." Note that this stack grows downward from a large index number to a small index number in the array, and must hold at most SIZE items.

```
#define SIZE 100
unsigned char stack[SIZE];
int sp;

void init_stack(void) {
    sp = SIZE;
}

int push(unsigned char data) {
    if( sp  (E)  ) /* stack overflow */
        return( -1 );
     (F)  ;
     (G)  = data;
     (H)  ;
    return( 1 );
}

int pop(void) {
    unsigned char data;
    if( sp  (I)  ) /* stack underflow */
        return( -1 );
     (J)  ;
    data =  (K)  ;
     (L)  ;
    return( data );
}

int depth(void) {
    return(  (M)  );
}
```

(4) The implementation of the stack in Question (3) has the restriction that the maximum number of items in the stack is fixed at compilation time. Describe a method to overcome this restriction. Note that you do not have to write a complete C program.

Write the answers to Problem II on the second answer sheet, and clearly label it at the top of the page as "Problem II."

Problem II Let $\mathcal{N}$ be the set of non-negative integers $\{0, 1, 2, \dots\}$. We define the recursive function $f : \mathcal{N} \rightarrow \mathcal{N}$ by:

$$\begin{aligned} f(0) &= 0, \\ f(1) &= 1, \text{ and} \\ f(n) &= f(n-1) + f(n-2) \quad (\text{if } n \geq 2). \end{aligned}$$

Select two questions from (1) to (3) below, and write their answers on the answer sheet.

- (1) We design a logical circuit whose input is a non-negative 3-bit value $(x_2x_1x_0)_2$, and output is $f((x_2x_1x_0)_2 \bmod 8) = (y_2y_1y_0)_2$, where $x \bmod y$ is the remainder after division of x by y when x and y are integers ($y \neq 0$). Answer the following questions.

- (a) Write the truth tables for the outputs y_2, y_1, y_0 .
- (b) Write the canonical sum-of-products forms of y_2 and y_1 .
- (c) Describe the Karnaugh map for y_0 , and show the simplest logical expression.

- (2) Answer the following questions.

- (a) For any $n \geq 2$, prove $f(n) < f(n+1)$ by mathematical induction.
- (b) If the function f is a surjective (onto) map, prove it; if it is not, give a counter example.
- (c) Show an element $m \in \mathcal{N}$ such that the function $g : \mathcal{N} \rightarrow \mathcal{N}$ defined below is injective.

$$\begin{aligned} g(m) &= 4, \text{ and} \\ g(n) &= f(n) \quad (\text{if } n \neq m). \end{aligned}$$

- (3) Answer the following questions.

- (a) If the expressions below are true, prove them; if they are false, give counter examples.
 - (i) $\forall n \in \mathcal{N} (\forall m \in \mathcal{N} \ f(n) = f(m) + f(m+1))$
 - (ii) $\forall n \in \mathcal{N} (\exists m \in \mathcal{N} \ f(3n) = 2m)$
- (b) Show all the sets $M \subset \mathcal{N}$ such that the logical expression $\forall n \in M \ (n = f(n))$ is true.